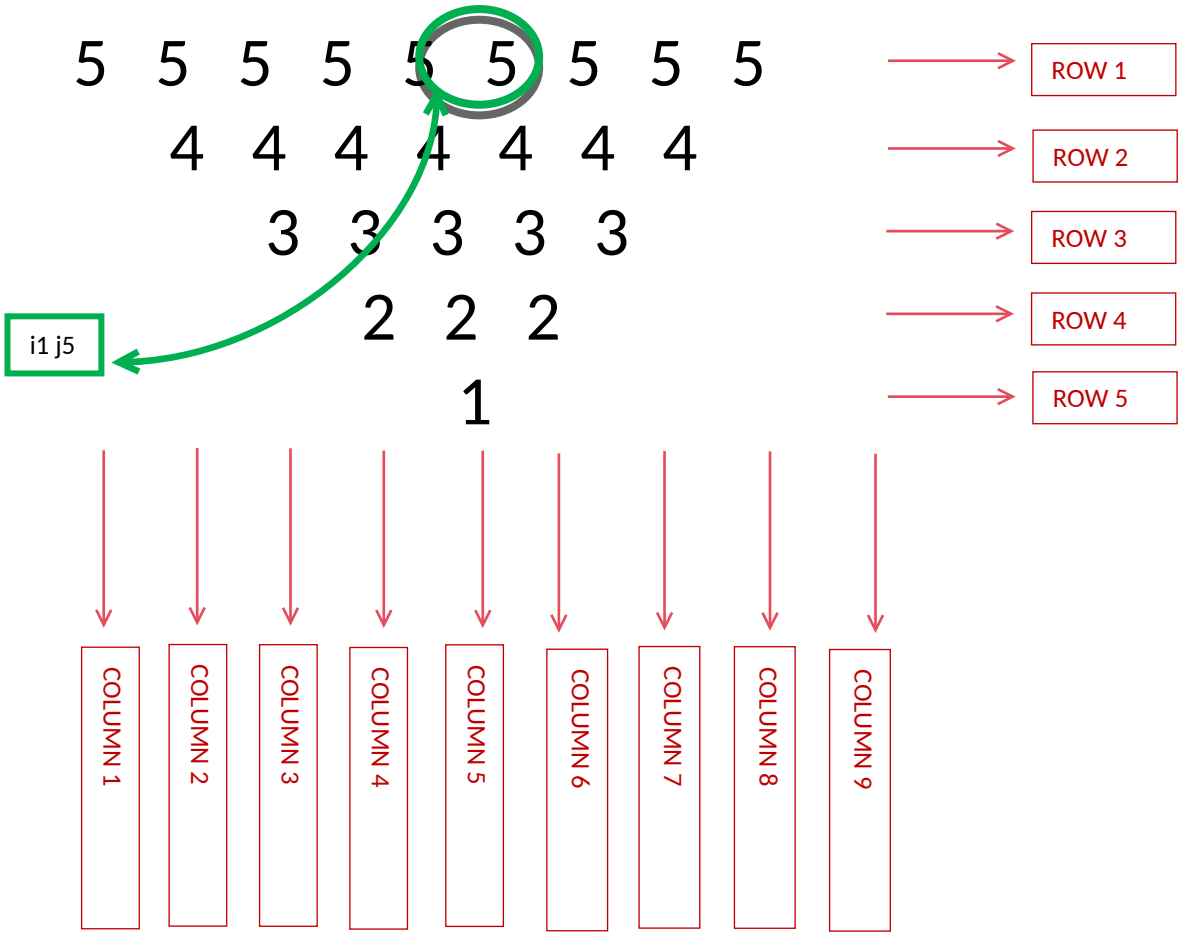
WEEK 01 MODULE
Programming Practice

TOPIC : 2 ‘ Number Pattern’

13. SAME NUMBER INVERTED PYRAMID



1. Problem Analysis :-

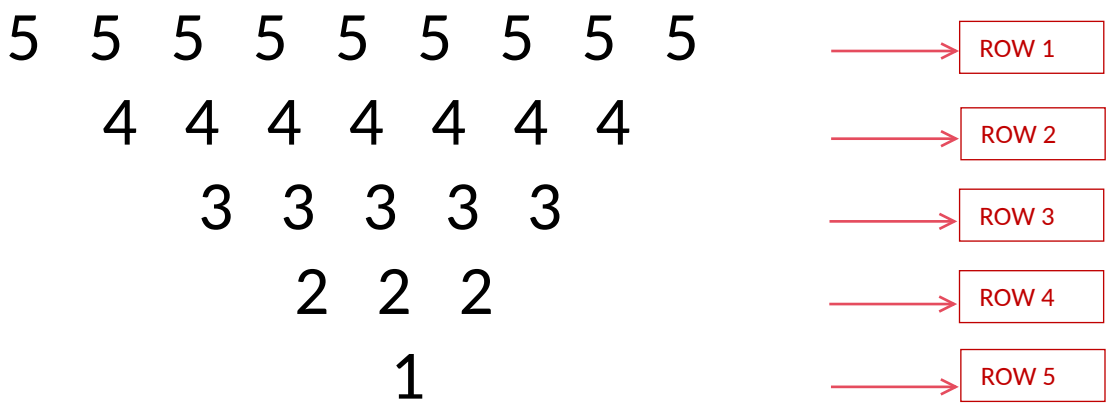
i. Pattern Structure:

- The output forms a centered inverted same number pyramid.

- Unlike the Number Pyramid, the pattern starts with the maximum width at the top and gradually shrinks toward the bottom.

- spaces are printed before the number so that the pattern appears inverted and centered.

- For n=5



ii. Execution Flow:

Step 1: Observe the Grid Concept:

- For n = 5

```
5 5 5 5 5 5 5 5 5
  4 4 4 4 4 4 4
    3 3 3 3 3
      2 2 2
        1
```

The pattern contracts/shrinks symmetrically from the center.

As we move downward:

- The number of spaces increases.
- The number count decreases by 2

Both of these changes occur simultaneously.

Therefore, this pattern requires two different calculations in every row.

- Each cell in the grid represents a unique coordinate (i, j), where i denotes the row index and j denotes the column index.

-The pattern can be viewed as two regions:

✦ **Region 1: Leading Spaces**

These spaces shift the number toward the center.

✦ **Region 2: Numbers**

These form the visible pyramid.

For n = 5

Row 1 : [ ][5 5 5 5 5 5 5 5 5]

Row 2 : [ ][4 4 4 4 4 4 4]

Row 3 : [ ][3 3 3 3 3]

Row 4 : [ ][2 2 2]

Row 5 : [][1]

As the row index increases:

- Spaces increase .
- number count decreases .

-> Row Analysis :-

Row Index (I)	1	2	3	4	5
Spaces	0	1	2	3	4
number	9	7	5	3	1
Printed value	5 5 5 5 5 5 5 5 5	4 4 4 4 4 4 4	3 3 3 3 3	2 2 2	1

The pyramid is controlled by two formulas:

Spaces = i

Number = 2 ×(n - i) - 1

Step 2: Traverse the grid row by row.

- The outer loop is responsible for generating rows

- for(int i = 0; i < n; i++)

Step 3: For every row, calculate the number of leading spaces required.

- The number of spaces depends on the current row index.
- The relationship is:

- Spaces = i

Step 4: Traverse the required number of positions and print number.

- This shifts the number toward the center.
- The number of spaces increases as the row index increases.

Step 5: After printing spaces, calculate the number required.

- The relationship is:

- Number = 2 ×(n - i) - 1

Step 6: Print value = n - i

- The number count decreases by 2 in every row.

Step 7: After all number of the current row have been printed, move the cursor to the next line

Step 8: Repeat the process for all rows until the outer loop terminates.

2. Condition Analysis

No conditional statements are required.

- Because every position visited by the space loop prints a space.

- Every position visited by the number loop prints a number.

- The pattern is controlled entirely through loop limits.

3. Core Logic Transformation

---

inverted same number pyramid Pattern:-

for(int j = 0; j < 2 * (n - i) - 1; j++)

- Meaning:

Stars decrease by 2 in every row.

Additionally :-

for(int j = 0; j < i ; j++)

- controls the alignment of the pattern.

---

4. Program :-

```
package week1_module;
import java.util.Scanner;
public class InvertedSameNumberPyramid {

	public static void main(String[] args) {
	Scanner sc = new Scanner(System.in);

	System.out.print("Enter the number of rows: ");
	int n = sc.nextInt();
		for(int i = 0; i < n; i++) {
			// Print spaces
			for(int j = 0; j <  i ; j++) {
				System.out.print(" ");
			}
			// Print stars
			for(int j = 1; j <= ( 2 * ( n - i ) - 1); j++) {
				System.out.print( ( n - i ) + " ");
			}

			System.out.println();
		}

		sc.close();
	}
}
```

For pyramids, using **0-based indexing (**i = 0**)** ie formulas become:

Spaces = i

Numbers = 2 * (n - i) - 1

Value = n - I

If you force **1-based indexing**, the formulas become :

Spaces = i - 1
Numbers = 2 * (n - i + 1) - 1
Value = n - i + 1

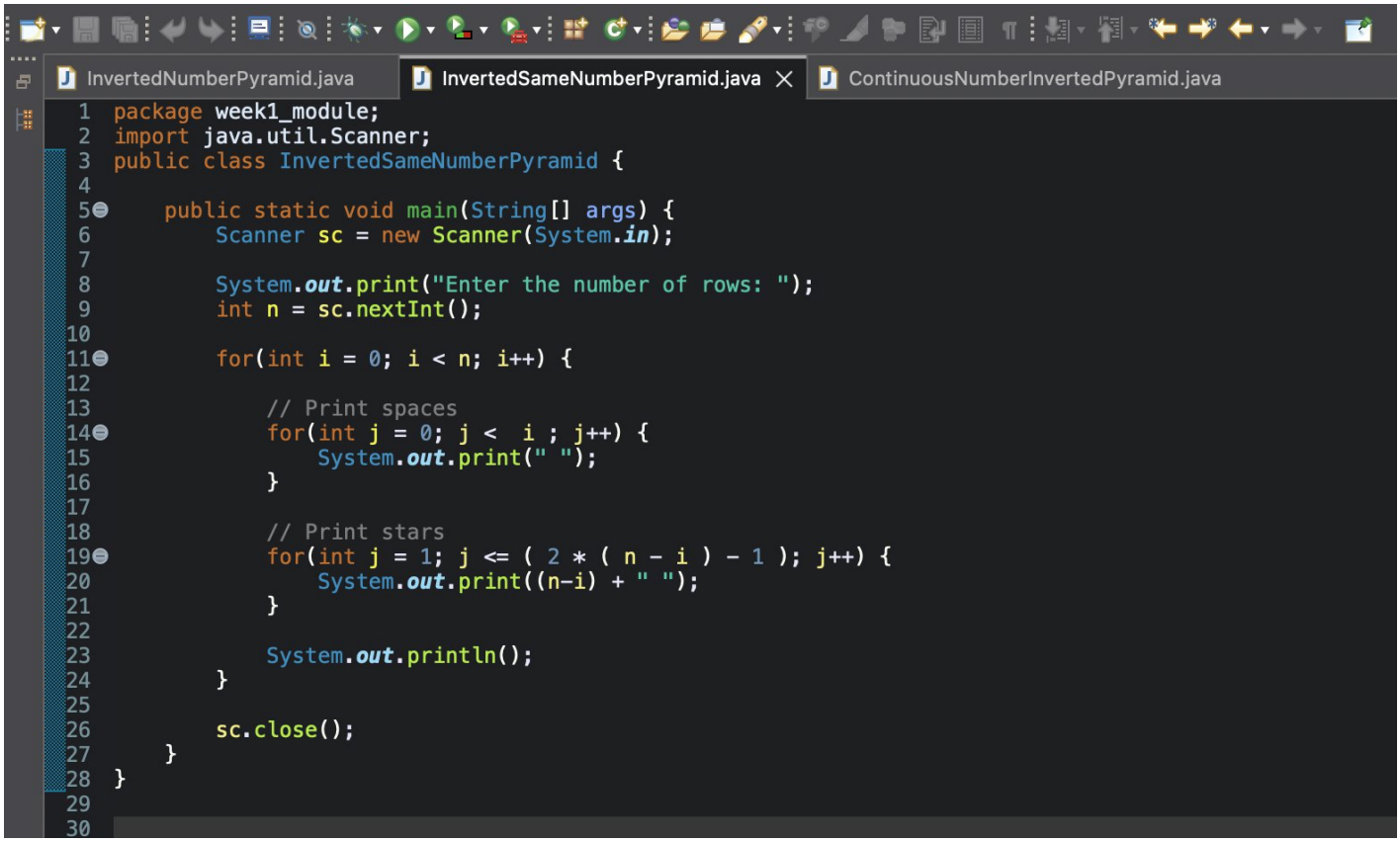
5. Program Output :-

Enter the number of rows: 5

```
5  5  5  5  5  5  5  5  5
   4  4  4  4  4  4  4
      3  3  3  3  3
         2  2  2
            1
```

6. **ScreenShot :-**

i. Program :-



ii. Program's Output :-